Sesiones y riesgos

Diapositivas de apoyo: Si toman tu sesión, actúan como tú y comparación de riesgos. Ensayar a 125-145 palabras por minuto. Usar una credencial temporal como analogía, aclarando sus límites.

La sesión mantiene la continuidad

Hasta ahora vimos datos que se convierten en instrucciones y acciones que pueden inducirse sin consentimiento. Mi parte se centra en lo que mantiene nuestra identidad entre una petición y la siguiente: la sesión. Cuando entramos a una plataforma, el servidor comprueba nuestras credenciales. Después necesitamos seguir navegando sin repetir la contraseña en cada paso. Una sesión permite relacionar esas acciones con una identidad o un contexto. Es una función normal de la web, pero su seguridad no termina cuando el inicio de sesión responde correctamente.

Podemos pensar en una credencial temporal para entrar a un evento. Al comenzar se verifica quiénes somos; durante la visita mostramos una credencial. Si alguien consigue una copia válida y el control de acceso solo comprueba esa credencial, podría hacerse pasar por nosotros. Esta comparación ayuda a entender el secuestro de sesión. No significa que todos los sistemas usen el mismo formato ni que cualquier cookie permita entrar. Hablamos de obtener o reutilizar una credencial que el sistema acepte para actuar dentro de una sesión.

Secuestro y fijación

El secuestro de sesión consiste en aprovechar una credencial válida de otra persona. La exposición puede producirse por fallos en el transporte, filtraciones, problemas en el navegador o errores de manejo, según la aplicación. No vamos a enseñar a capturar credenciales ni a reutilizar cuentas ajenas. Lo importante es comprender la consecuencia: aunque la contraseña no cambie, otra persona podría actuar con una sesión que sigue siendo aceptada. Por eso cambiar una contraseña y terminar todas las sesiones son operaciones relacionadas, pero no necesariamente equivalentes.

La fijación de sesión es distinta. En lugar de esperar a obtener una credencial después del acceso, el atacante intenta que la víctima utilice un identificador conocido antes de autenticarse y que ese identificador conserve la sesión autenticada. Podemos compararlo con dejar preparada una pulsera y lograr que después se habilite sin sustituirla. En sistemas con identificadores de sesión almacenados en el servidor, renovar ese identificador al autenticar reduce este riesgo. La implementación concreta depende del sistema; no todos los modelos de sesión funcionan de la misma manera.

Libreto de la comparación: Una credencial en otras manos.

En las dos ventanas aparece la misma credencial ficticia. A la izquierda la envía el navegador de Ana para continuar una sesión que le pertenece. A la derecha la presenta otra persona desde otro navegador. La animación parte de que esa persona ya dispone de una copia; no muestra ni explica cómo se obtuvo.

Después del clic, las dos ventanas reciben Bienvenida, Ana. En el lado normal es el resultado esperado. En el otro lado significa que alguien ajeno puede entrar con la identidad de Ana. El servidor reconoce la credencial válida, no a la persona físicamente sentada frente al navegador.

No se escribió una contraseña ni se adivinó una cuenta. El problema es que una copia todavía aceptada conserva la sesión. Por eso debemos proteger la credencial, limitar cuánto dura e invalidarla cuando corresponda. La siguiente explicación de HTTPS, cookies y ciclo de vida responde a ese riesgo.

Proteger el transporte y el navegador

La primera capa es HTTPS. Cifra la comunicación y ayuda a protegerla frente a observadores en tránsito, dentro de su modelo de confianza. Para las cookies sensibles, el atributo Secure limita su envío a conexiones HTTPS. Ambos controles deben acompañarse: no basta con mostrar una dirección segura en una pantalla si el resto del flujo permite usar la credencial de manera insegura. En el laboratorio trabajaremos sobre HTTP de loopback, solo en la misma computadora. Esa excepción de demostración no debe copiarse como una configuración de producción.

La segunda capa incluye HttpOnly. Este atributo impide que la cookie se lea mediante el acceso habitual de JavaScript a las cookies. Es útil para reducir un camino de exposición, pero no hace desaparecer XSS. Un script que ya se ejecuta en una página puede interactuar con sus elementos o realizar acciones permitidas a esa sesión. Recordemos la diferencia entre leer la credencial y utilizar el navegador de la persona para actuar. Por eso no presentaremos HttpOnly como una garantía absoluta frente a todos los ataques a usuarios.

SameSite, que acabamos de conocer, limita determinados envíos de cookies entre sitios. Es especialmente útil como capa frente a solicitudes inducidas, aunque no reemplaza los tokens CSRF donde corresponden. Los atributos funcionan juntos y deben elegirse según el uso legítimo de la aplicación. Una configuración extremadamente restrictiva puede interferir con algunos flujos; una demasiado permisiva puede dejar riesgos. El trabajo consiste en entender el flujo esperado y protegerlo, no en copiar una lista de palabras sin comprobar qué hacen.

Tiempo, cierre y revocación

Una credencial de sesión no debería durar para siempre. Hay que definir una duración, comprobarla y considerar la inactividad cuando el diseño lo requiere. En operaciones sensibles puede pedirse una nueva autenticación. La idea es limitar durante cuánto tiempo una credencial expuesta podría seguir siendo útil. No existe una duración universal correcta para todos los productos. Una plataforma con información delicada y una aplicación de preferencias públicas pueden necesitar decisiones diferentes. Lo importante es que exista una política explícita y que el servidor la haga cumplir.

Cerrar sesión también merece atención. Eliminar una cookie del navegador termina la continuidad en ese navegador, pero no necesariamente invalida una copia que alguien hubiera obtenido. En un sistema con sesiones en servidor, revocar el registro o el identificador permite dejar de aceptarlo. Otros modelos requieren estrategias de expiración o revocación específicas. No debemos prometer una revocación completa si solo borramos un valor local. Esta diferencia explica por qué algunas aplicaciones ofrecen cerrar otras sesiones o revisar dispositivos conectados como funciones distintas de salir en la pestaña actual.

En Flask, la sesión estándar se guarda en una cookie firmada. Firmada significa que el servidor puede detectar modificaciones no válidas; no significa que su contenido esté cifrado. Por eso no se deben guardar contraseñas ni secretos allí. La clave usada para firmar debe protegerse y no incluirse en un repositorio público. También debemos distinguir este modelo de una sesión cuyo contenido vive en el servidor y cuyo navegador conserva un identificador. Las recomendaciones de renovación del identificador deben aplicarse al modelo real, no repetirse sin contexto.

Comparar los riesgos sin confundirlos

Ahora miremos la tabla de riesgos. SQLi se relaciona con la interpretación de una consulta por la base de datos. XSS se relaciona con contenido ejecutable en el navegador. CSRF se relaciona con una acción inducida que aprovecha credenciales enviadas automáticamente. El secuestro de sesión se relaciona con actuar mediante una credencial válida ajena. Estos problemas pueden combinarse en un sistema, pero describen causas distintas. Distinguirlas permite elegir controles que atienden la raíz y no solamente un síntoma visible.

Para cerrar mi parte, propongo cuatro preguntas sencillas: ¿cómo protegemos la credencial mientras viaja?, ¿quién puede leerla?, ¿cuánto tiempo es válida?, ¿cómo dejamos de aceptarla? Si una aplicación responde bien esas preguntas, su gestión de sesión tiene una base más clara. A continuación veremos el ejemplo local en Flask y una lista de prevención por capas. La demostración permitirá conectar estas ideas con decisiones visibles en el código y en el navegador, manteniendo siempre el alcance educativo de la actividad.

Apoyo para comprender y ensayar

Firmar permite comprobar integridad y autenticidad bajo una clave; cifrar busca ocultar el contenido. Revocar es dejar de aceptar una credencial, no simplemente ocultarla en la interfaz. Si surge la pregunta sobre doble factor, explicar que mejora la autenticación, pero una sesión válida sigue requiriendo protección. Hacer una pausa tras la analogía y otra antes de distinguir la cookie firmada de Flask. No mostrar credenciales reales en la exposición.

Indicaciones para presentar esta parte

Después de la explicación de sesiones, avanzar a la comparación. Señalar primero quién utiliza cada navegador y luego que las credenciales son iguales. Esperar los dos saludos de Ana. No presentar el ejemplo como una demostración de captura de cookies: la copia ya existe al comenzar. Seguir con la comparación de riesgos.

Diccionario de términos

Sesión: Continuidad que relaciona varias solicitudes con una identidad o contexto.

Secuestro de sesión: Uso indebido de una credencial válida de otra persona.

Fijación de sesión: Intento de conservar, tras autenticar, un identificador conocido antes por el atacante.

HTTPS: HTTP protegido mediante TLS para cifrar la comunicación en tránsito.

Secure: Atributo de cookie que limita su envío a HTTPS.

HttpOnly: Atributo que impide leer la cookie mediante el acceso habitual de JavaScript.

Firma: Comprobación criptográfica de autenticidad e integridad; no oculta el contenido.

Cifrado: Transformación destinada a ocultar información a quienes no poseen la clave apropiada.

Revocación: Decisión de dejar de aceptar una credencial que antes era válida.